

Sales Report Generator

1. Executive Summary

The Sales Report Generator is a Java command-line application designed to read transaction data from an external CSV file, aggregate metrics by product category, calculate summary statistics, and format outputs for console display and file generation.

2. Team Responsibilities & File Ownership

Team Member	Responsibilities	Assigned Files
Member 1 L.Kajaani SE/2023/049	CSV parsing logic, data model construction, financial calculation methods, and input dataset maintenance.	• ProductSale.java • CsvReader.java • SalesReader.java • SalesReport.java • SalesCalculator.java • data/sales.csv
Member 2 M.Megala SE/2023/030	Output strategy pattern design, unit test suites, and Maven project build specifications.	• OutputStrategy.java • ConsoleOutput.java • FileOutput.java • SalesCalculatorTest.java • pom.xml
Member 3 C.Divya SE/2023/046	CLI execution entry point, custom application exception handling, repository configurations, and documentation.	• SalesReporter.java • SalesReportException.java • README.md • .gitignore

3. Directory Structure

SalesReporter/

data/

sales.csv — Input sales dataset (Member 1)

src/main/java/

exception/

SalesReportException.java — Custom Exception Handler (Member 3)

model/

ProductSale.java — Sales Data Model (Member 1)

output/

OutputStrategy.java — Output Interface (Member 2)

ConsoleOutput.java — Terminal Output Writer (Member 2)

FileOutput.java — File Output Writer (Member 2)

reader/

CsvReader.java — Base CSV Reader (Member 1)

SalesReader.java — Data Reader Service (Member 1)

report/

SalesReport.java — Sales Report Model (Member 1)

service/

SalesCalculator.java — Processing Math Engine (Member 1)

SalesReporter.java — Main Entry Point (Member 3)

src/test/java/service/

SalesCalculatorTest.java — Unit Tests (Member 2)

pom.xml — Build Config (Member 2)

README.md — Project Documentation (Member 3)

.gitignore — Git Exclusions (Member 3)

4. Key Design Implementations

Strategy Pattern: The project uses the OutputStrategy interface to separate formatting and target delivery (console vs. file), allowing new output destinations to be added without modifying core execution logic.

Custom Exception Handling: The SalesReportException is used to handle invalid command-line options, such as an unsupported output method or a missing output file path. Standard IOException and IllegalArgumentException are also handled appropriately.

Package Modularization: Classes are organized into focused packages (model, reader, service, output, and exception) to maintain strict separation of concerns.

5. Verification and Build Details

- **Build Environment:** Maven
- **Test Class:** SalesCalculatorTest
- **Execution Command:** mvn clean test
- **Result Status:** BUILD SUCCESS (All unit tests verified)